Kong

LIVE HANDS-ON WEBINAR

Kong Webinar Series

BROADCAST STATUS

Starting Soon

We will begin in just a few minutes.
Get ready for a hands-on session

Kong

Question

The AI threat on your systems?

How long does it take your organisation, from the moment a security vulnerability is disclosed, to have someone actively working on it?

Kong

LIVE HANDS-ON WEBINAR

Hardening Konnect

Security practices that protect your API platform.
The real-world attacks they stop.

Presented By

Mayank Kakani

Senior Technical Customer Success

FOR CUSTOMER DEV & OPERATIONS TEAMS

24

CONTROLS

ACROSS 6 SECURITY DOMAINS

- TLS & Network
- Secrets & Vaults
- RBAC
- Gateway Hardening
- Observability
- Resilience

TODAY'S AGENDA

What We're Covering

01

Why This Matters

API attack stats + real-world breaches that cost organizations millions

02

24-Control Framework

6 security domains covering your full Konnect deployment surface

03

Labs

Hands-on labs covering controls and hardening scenarios

04

Q&A

Open interactive session to address team queries and deployments

** Hands-on labs run throughout each section to ensure practical application of concepts*

WHY THIS MATTERS — THE THREAT LANDSCAPE

APIs Are the #1 Attack Surface

92%

of IT leaders rank AI-enhanced threats as their #1 API security concern
— Kong API Security Perspectives, 2025

$5.72M

average cost of an AI-powered breach
— vs $4.45M for non-AI incidents
— IBM Cost of a Data Breach Report, 2025

98.9%

of AI-related vulnerabilities are API-based. AI agents communicate through APIs — and attackers know it
— Wallarm ThreatStats, 2025

The Common Thread — and Why AI Makes It Worse:

- Exposed admin interfaces with no network restriction
- Credentials stored in config files or environment without a secrets manager
- No rate limiting — enabling brute-force and credential stuffing attacks
- No audit logging — attackers, including AI agents, dwell undetected for weeks

THE EMERGING THREAT — AI-ENHANCED API ATTACKS

AI Didn't Create the Vulnerabilities. It Just Made Them Faster.

51%

of developers cite unauthorized API calls from AI agents as their #1 security concern
— Postman State of the API, 2025

65%

of organizations say generative AI poses a serious to extreme risk to their APIs
— Traceable AI, 2025 Global State of API Security

1.2 hrs

average time for an AI-driven attack to fully exploit a vulnerability — down from days
— SQ Magazine / multiple sources, 2025

Agentic AI Attacks

Only 37% of organizations using AI agents have dedicated API security in place. Each agent is a new identity making API calls — often with long-lived shared credentials and no rate limiting.
— Salt Security H2 2025

AI-Accelerated Exploit Dev

Attackers now scan for new API vulnerabilities within 15 minutes of disclosure. AI-generated exploit code cuts time-to-attack from weeks to hours. 72% YoY increase in AI-assisted attacks.
— Multiple sources, 2025

AI Code Introduces New APIs

62% of organizations use GenAI in API development. AI-generated code routinely ships undocumented endpoints, weak auth, and shadow APIs — expanding attack surface faster than teams can audit.
— Salt Security H2 2025

REAL ATTACK — CASE STUDY 01

The Open Front Door: Exposed Admin Interfaces

⚠ REAL ATTACK

2017: 28,000 Databases Ransomed in 48hrs

The attack was trivially simple:

- MongoDB admin ports left exposed to the internet
- No authentication on management interface
- HTTP only — credentials transmitted in cleartext

Attackers automated the scan, wiped data, and left ransom notes. 28,000 databases. 48 hours.

Same attack pattern applies to any exposed API gateway admin API.

⚠ REAL ATTACK

2022 Optus Breach — 10M Records

An unauthenticated API endpoint was left open — no token, no IP restriction. Attackers iterated through customer IDs sequentially. 10 million records leaked. No rate limiting to slow them down.

KONG KONNECT CONTROL

Kong Hardening Controls that stop this:

#1 Disable HTTP across all Kong components
#11 Restrict Admin API at network level (IP allowlist)
#7 RBAC with least-privilege roles on every workspace
#24 Global rate limiting on all services

REAL ATTACK — CASE STUDY 02

Secrets in Plain Sight: Credentials That Should Have Never Been There

⚠ REAL ATTACK

2019 Capital One — 106M Records

A server-side request forgery (SSRF) flaw in a WAF exposed the AWS metadata endpoint — which handed over IAM credentials. The attacker walked straight in.

Root cause: credentials available in environment/config without vault protection.

⚠ REAL ATTACK

2021 Twitch Source Code Breach

Internal API keys and hardcoded secrets were found embedded in source code. Once one credential was exposed, attackers pivoted to internal systems.

Root cause: no secrets manager — keys lived in config files and git history.

⚠ REAL ATTACK

POODLE 2014 / BEAST 2011: TLS Downgrade Attacks

Attackers forced connections to downgrade to SSLv3 / TLS 1.0, then decrypted session cookies. Any service supporting old TLS versions was vulnerable.

Fix: enforce TLS 1.2+ minimum.

KONG KONNECT CONTROL

Kong Hardening Controls that stop this:

#2 LDAP password via secrets manager, not kong.conf
#3 Consumer credential secrets (key-auth) via Vault
#8 Key-auth encrypted and stored in Vault
#15 TLS minimum version enforced at 1.2

THE KONG HARDENING FRAMEWORK

6 Security Domains — 24 Controls

1 Network & Transport Security

4 controls
Disable HTTP · Admin API restriction
TLS for all connections · Min TLS 1.2

2 Secrets & Credential Mgmt

4 controls
LDAP + consumer creds in Vault
Key-auth encrypted · No anon reporting

3 Authentication & Access

2 controls
RBAC with least privilege
DP nodes isolated at network level

4 Gateway Hardening

5 controls
Fingerprinting off · Debug header off
Nginx logs off · Real IP · Lua sandboxed

5 Observability & Defense

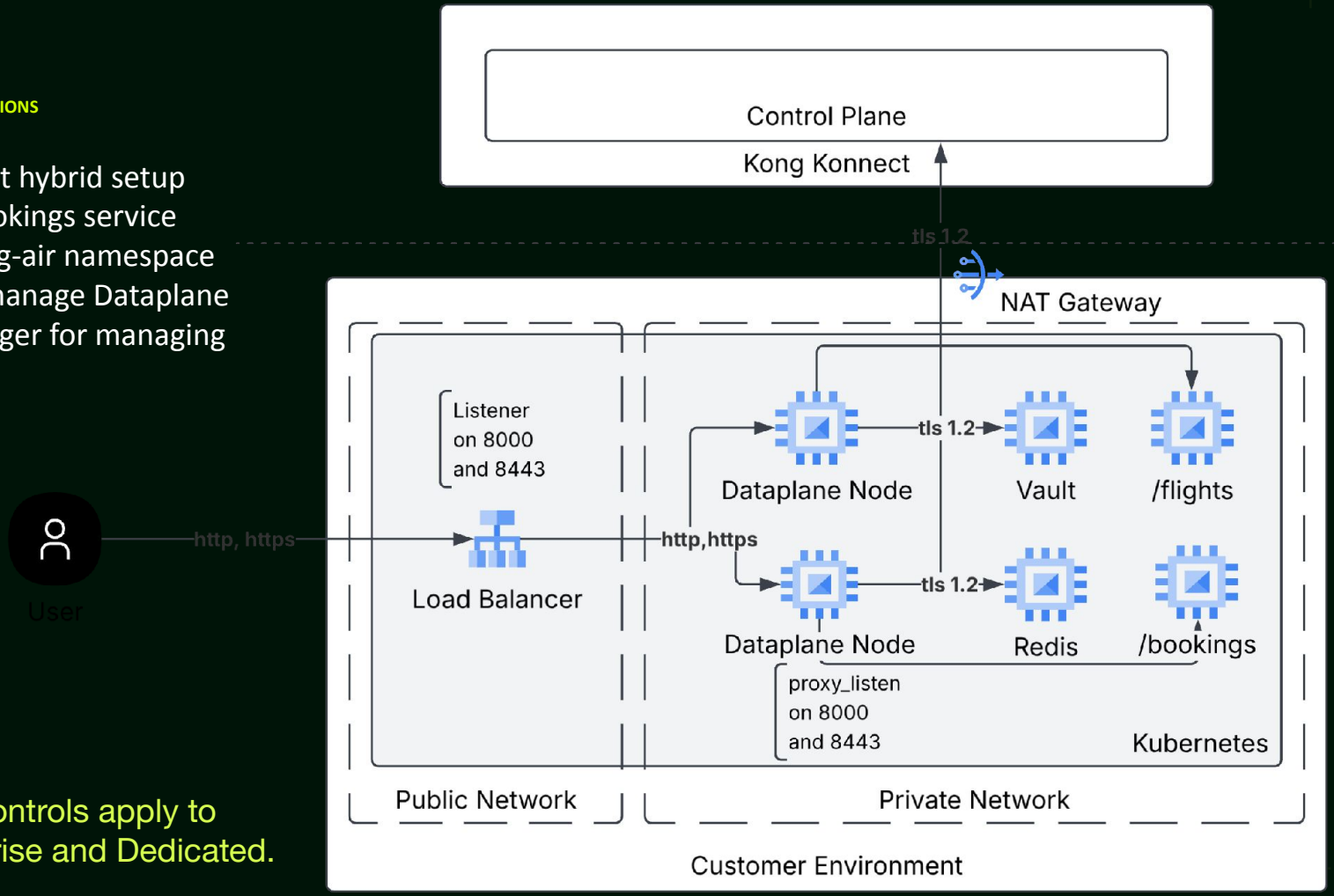
6 controls
CP/DP health checks · Vitals monitoring
Audit logs · Global rate limiting

6 Resilience & Infrastructure

3 controls
DP cert stored securely
HA DP nodes · 2+ per region

CURRENT STATE & CONSIDERATIONS

1. Running Konnect hybrid setup
2. /flights and /bookings service deployed in kong-air namespace
3. Using Helm to manage Dataplane
4. Using CertManager for managing Certificates.



Same security controls apply to Konnect, Enterprise and Dedicated.

DOMAIN 01 — NETWORK & TRANSPORT SECURITY

Secure the Wire Before Anything Else

#1 **Disable HTTP Everywhere**

Data plane, Dev Portal — all must be HTTPS. HTTP anywhere in your stack is a credential interception waiting to happen.

#11 **Restrict Admin API at Network Level**

Konnect org-level IP allowlisting restricts UI and API access to your corporate network. Separately, verify your DP admin port (8444) stays bound to 127.0.0.1 — confirm it is never exposed publicly.

#14 **TLS for All Integrations**

Database, cache, and all third-party connections must use TLS. HTTP integrations create man-in-the-middle risk.

#15 **Enforce TLS 1.2+ Minimum**

Disable SSL 3.0, TLS 1.0, and TLS 1.1. POODLE and BEAST are well-documented, automated attacks.

⚠ REAL ATTACK

Real Attack Saved: Firesheep & HTTP Session Hijacking

In 2010, a Firefox plugin called Firesheep captured unencrypted session cookies on any WiFi network. HTTP API calls made it trivial to hijack authenticated sessions. Any API gateway still running HTTP faces the same risk today.

HTTPS everywhere eliminated this class of attack completely.

HANDS-ON LAB →

Set KONG_SSL_PROTOCOLS and KONG_PROXY_LISTEN env vars on your DP container
Test TLS enforcement with openssl s_client against the DP

HANDS-ON LAB 01 — NETWORK & TLS

Enforce TLS Across Konnect

```
# Konnect DP uses environment variables
# Set these in docker-compose.yml or Helm values.yaml → env:

KONG_PROXY_LISTEN: "0.0.0.0:8443 ssl reuseport backlog=16384"
# Do NOT add 8000 — HTTP listener must stay disabled

# TLS minimum version
KONG_SSL_PROTOCOLS: "TLSv1.2 TLSv1.3"
KONG_SSL_CIPHERS:
"ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256"

# Test after DP restarts:
# openssl s_client -connect <dp-host>:8443 -tls1_1  → should FAIL
# openssl s_client -connect <dp-host>:8443 -tls1_2  → should SUCCEED
```

Lab Steps

1. Edit your DP container env vars — docker-compose.yml or Helm values.yaml
2. Confirm KONG_PROXY_LISTEN does not include port 8000 — HTTP must be absent
3. Set KONG_SSL_PROTOCOLS="TLSv1.2 TLSv1.3" in the DP env section
4. Redeploy the DP container and verify with openssl s_client on port 8443
5. Confirm TLS 1.1 connection is rejected (expected: handshake failure)

✓ VERIFY: KONG_SSL_PROTOCOLS SET + OPENSSL S_CLIENT -TLS1_1 → HANDSHAKE_FAILURE

DOMAIN 02 — SECRETS & CREDENTIAL MANAGEMENT

Zero Cleartext Credentials. Ever.

#2 **LDAP Password via Secrets Manager**

Never hardcode your LDAP bind password in a DP environment variable in plaintext. Source it from Vault using the {vault://hcv/...} reference syntax in your KONG_LDAP_ATTRIBUTE env var.

#3 **Consumer Credential Secrets via Vault**

API keys and JWT secrets issued to API consumers should not live as plaintext values in plugin config. Reference them from a Vault backend configured in Konnect — Gateway Manager → Vaults.

#8 **Key-Auth: Encrypted + Vault-Backed**

API keys stored in Kong should be encrypted at rest. Reference them from your Vault — don't embed them in plugin config.

#10 **Disable Anonymous Reporting**

Kong's default phone-home telemetry sends deployment metadata externally. Disable it in production environments.

⚠ REAL ATTACK

Thousands of API Keys Leak on GitHub Daily

GitHub's secret scanning program detects millions of leaked credentials each year. Hardcoded database passwords, API keys, and LDAP credentials committed to repositories are found and exploited within hours of exposure.

A secrets manager adds one layer between an attacker and your database.

Supported Vault Backends in Konnect:

HashiCorp Vault · AWS Secrets Manager · GCP Secret Manager · Azure Key Vault · Environment Variables

HANDS-ON LAB → Configure HashiCorp Vault as Kong secret backend

HANDS-ON LAB 02 — SECRETS & VAULT

Store Zero Plaintext Credentials in Konnect

```
# Konnect Admin API — Create Vault backend
export KONNECT_PAT="kpat_your_token"
export REGION="us"    # us / eu / au / ap
export CP_ID="your-control-plane-id"

curl -sX POST \

"https://$REGION.api.konghq.com/v2/control-planes/$CP_ID/core-entities/
vaults" \
  -H "Authorization: Bearer $KONNECT_PAT" \
  -H "Content-Type: application/json" \
  -d '{"name":"hcv","prefix":"hcv","config":{"protocol":"https",
    "host":"vault.internal.example.com","port":8200,
    "mount":"secret","kv":"v2","token":"hvs.YOUR_TOKEN"}}'

# DP env vars (docker-compose.yml / Helm values.yaml → env:):
KONG_VAULT_HCV_PROTOCOL: "https"
KONG_VAULT_HCV_HOST:     "vault.internal.example.com"
KONG_VAULT_HCV_PORT:     "8200"
KONG_VAULT_HCV_TOKEN:    "hvs.YOUR_TOKEN"
KONG_VAULT_HCV_KV:       "v2"
KONG_VAULT_HCV_MOUNT:    "secret"

# Reference secrets in plugin config:
# secret-value: "{vault://hcv/kong/ldap-bind-pw}"

# Verify via API:
# GET /core-entities/vaults | jq '.data[]|{prefix,name}'
```

Lab Steps

1. Create Vault backend via Konnect Admin API (POST /core-entities/vaults — see command on left)
2. Set KONG_VAULT_HCV_* env vars on your DP container
3. In plugin config, replace any plaintext secret value with {vault://hcv/path/to/secret}
4. Redeploy the DP container — verify vault appears via API: GET /core-entities/vaults
5. Confirm zero plaintext credentials in DP env vars — vault references only
6. Test: revoke Vault token — Kong should fail gracefully, not log the secret

DOMAIN 03 — AUTHENTICATION & ACCESS CONTROL

Who Gets In, and How Far They Go

#7

RBAC: Principle of Least Privilege

Konnect RBAC is managed via Organization → Teams in the Konnect UI. Scope roles to specific Control Planes — not org-wide. Available roles: Org Admin, Control Plane Admin, Viewer, and Custom Roles.

Recommended role hierarchy:

- Org Admin — account-level only
- Control Plane Admin — scoped per CP
- Viewer — developers and auditors
- Custom Roles — granular permissions

Enable MFA for users

#9

Isolate DP Nodes at Network Level

Run DP nodes in a private VPC or subnet. Only your load balancer should face external traffic. Block direct inbound access to DP ports 8000/8443 from the internet. Your management port (8444) must be accessible only from your ops network.

⚠ REAL ATTACK

2022 Optus: RBAC Gaps Enable Mass Enumeration

The exposed Optus endpoint had no authentication at all. In Konnect, RBAC is enforced via Teams. A user with Viewer role on a Control Plane cannot modify services, routes, or plugins — even if their credentials are compromised.

Least privilege is your blast radius control.

HANDS-ON LAB 03 — ACCESS CONTROL

Configure RBAC with Least Privilege in Konnect

```
# Konnect RBAC — managed via Konnect API (NOT localhost:8001)
# Generate PAT: Konnect UI → Account Settings → Personal Access Tokens

export KONNECT_PAT="kpat_your_token_here"
export KONNECT_BASE="https://global.api.konghq.com"
export CP_ID="your-control-plane-id"   # from Konnect UI

# Create a scoped team
curl -sX POST "$KONNECT_BASE/v2/teams" \
  -H "Authorization: Bearer $KONNECT_PAT" \
  -H "Content-Type: application/json" \
  -d '{"name":"cp-readonly","description":"Developer read-only
access"}'

# Assign Viewer role scoped to one Control Plane only
curl -sX POST "$KONNECT_BASE/v2/teams/{team_id}/assigned-roles" \
  -H "Authorization: Bearer $KONNECT_PAT" \
  -H "Content-Type: application/json" \
  -d
'{"role_name":"viewer","entity_type_name":"control-planes","entity_id":
"'$CP_ID'"}'
```

Lab Steps

1. Generate a PAT in Konnect UI (Account Settings → Personal Access Tokens)
2. Verify Org Admin assignments via API: curl -s '$KONNECT_BASE/v2/users' -H 'Authorization: Bearer $KONNECT_PAT' | jq '.data[] | {name,active}'
3. Create a 'cp-readonly' team via the Konnect API with Viewer role scoped to one CP
4. Invite a developer user and assign them to the cp-readonly team only
5. Log in as that developer — confirm they cannot modify services or plugins
6. Confirm no org-wide roles — all assignments scoped to specific control planes

Kong

DOMAIN 04 — GATEWAY HARDENING

Don't Advertise Your Stack to Attackers

#12 Disable Server Fingerprinting

Set KONG_HEADERS=off on your DP container. Every header advertising your gateway version is a CVE lookup invitation — remove them all.

#13 Disable the Debug Header

Kong-Debug:1 in a request triggers internal routing data in the response. Disable globally via the Konnect Admin API — POST a request-transformer plugin to /core-entities/plugins with config.remove.headers=[Kong-Debug].

#16 Disable Nginx Access Logs on DP

Set KONG_NGINX_HTTP_ACCESS_LOG=off on DP containers. Nginx access logs lack Kong's header masking — auth tokens can appear in cleartext.

#17 Configure Real IP

Set KONG_REAL_IP_HEADER, KONG_REAL_IP_RECURSIVE, and KONG_TRUSTED_IPS on your DP. Without this, rate limiting and ACLs operate against your LB's IP, not the real client.

#20 Disable Untrusted Lua Execution

Set KONG_UNTRUSTED_LUA=off on your DP. The pre/post function plugin runs unsandboxed Lua — in Konnect, control who can create plugins via RBAC roles.

⚠ REAL ATTACK

OWASP Security Misconfiguration (#5 of Top 10)

Version headers and debug endpoints are the first thing automated scanners look for. When your gateway announces "Kong/3.2.1", attackers run known CVE exploits against that exact version.

Every 2024 version-fingerprinting audit found over 60% of production gateways still serving Kong version in response headers.

What attackers see without hardening:

```
Server: openresty/1.21.4.3
X-Kong-Proxy-Latency: 12
Via: 1.1 kong/3.2.1
```

HANDS-ON LAB 04 — GATEWAY HARDENING

Remove Fingerprints. Kill the Debug Header.

```
# Konnect DP — environment variables
# Set in docker-compose.yml or Helm values.yaml → env:

KONG_HEADERS: "off"                    # strips Server, X-Kong-* headers
KONG_NGINX_HTTP_SERVER_TOKENS: "off"   # strips OpenResty version

KONG_NGINX_HTTP_ACCESS_LOG: "off"      # DP only — kills cleartext token
logging

KONG_REAL_IP_HEADER: "X-Forwarded-For"    # match your LB header
KONG_REAL_IP_RECURSIVE: "on"
KONG_TRUSTED_IPS: "10.0.0.0/8,172.16.0.0/12"  # your LB CIDR

# Disable Kong debug header — global plugin via Konnect Admin API:
curl -sX POST \

"https://$REGION.api.konghq.com/v2/control-planes/$CP_ID/core-entities/
plugins" \
  -H "Authorization: Bearer $KONNECT_PAT" \
  -H "Content-Type: application/json" \
  -d '{"name":"request-transformer","enabled":true,
    "config":{"remove":{"headers":["Kong-Debug"]}}}'

# Verify after DP redeploy:
# curl -I https://<dp-host>:8443/ | grep -iE 'server|kong|via' → no
output
```

Lab Steps

1. Add KONG_HEADERS=off and KONG_NGINX_HTTP_SERVER_TOKENS=off to DP env, redeploy
2. Run: curl -I https://<host>:8443/ | grep -iE 'server|kong|via'
3. Confirm: curl -I | grep -iE 'server|kong|via' returns nothing — no fingerprinting headers
4. Add request-transformer plugin via Konnect Admin API: POST /core-entities/plugins (command on left)
5. Add KONG_NGINX_HTTP_ACCESS_LOG=off to DP env and redeploy
6. Set KONG_REAL_IP_HEADER, KONG_TRUSTED_IPS to match your LB — redeploy and verify client IPs in access logs

Kong

DOMAIN 05 — OBSERVABILITY & DEFENSE

You Can't Protect What You Can't See

#4 Monitor DP Connection Status

Check DP↔CP connectivity via the Konnect Admin API: GET /v2/control-planes/{cp_id}/nodes — returns each DP node's status and last_seen timestamp. A 'disconnected' DP stops receiving config updates. Set alerting on status changes.

#5 DP Health Checks

Health check DP nodes at the LB layer. A DP failing silently continues to serve cached routes — until it doesn't.

#18 Konnect Analytics → Monitoring

Export traffic metrics via the Konnect Analytics API to Datadog, Grafana, or Splunk. Konnect Analytics provides per-service, per-route, and per-consumer breakdowns. Note: Grafana dashboard #7424 is for self-managed Kong only — it does not work with Konnect.

#19 Konnect Audit Logs — Always On

All admin actions are captured automatically. Export to your SIEM via the Konnect Admin API: PUT https://global.api.konghq.com/v2/audit-log-webhook with your SIEM endpoint and log_format. Retrieve logs via GET /v2/audit-logs.

#23 Global Observability Plugins

Audit global plugin coverage via Admin API: GET /core-entities/plugins?size=100 and filter where route and service are null. Any workspace with gaps in logging or tracing has a blind spot.

#24 Global Rate Limiting

Create via Admin API: POST /core-entities/plugins with name=rate-limiting and no service/route scope. Verify it's global: GET /core-entities/plugins and confirm the rate-limiter entry shows null for both service and route.

⚠️ *Uber 2016 Breach: attackers had access for 12+ months before detection. No audit logging = no forensic trail. 57 million records.*

HANDS-ON LAB 05 — OBSERVABILITY & DEFENSE

Global Rate Limiting + Audit Logs

```
# Auth vars — set once, reuse across all calls:
export PAT="kpat_your_token"
export RL="https://$REGION.api.konghq.com/v2/control-planes/$CP_ID"
export GL="https://global.api.konghq.com"
AUTH="-H 'Authorization: Bearer $PAT' -H 'Content-Type: application/json'"

# 1. Global rate-limiting plugin
curl -sX POST "$RL/core-entities/plugins" -H "Authorization: Bearer $PAT" \
  -H "Content-Type: application/json" \
  -d '{"name":"rate-limiting","config":{"minute":200,"hour":5000,
  "policy":"redis","redis":{"host":"redis.internal","port":6379}}}'

# 2. Audit Log SIEM export
curl -sX PUT "$GL/v2/audit-log-webhook" -H "Authorization: Bearer $PAT" \
  -H "Content-Type: application/json" \
  -d '{"endpoint":"https://siem.example.com/ingest","enabled":true,"log_format":"cef"}'

# 3. DP node health + recent audit events
curl -s "$RL/nodes" -H "Authorization: Bearer $PAT" | jq '.data[]|{hostname,status}'
curl -s "$GL/v2/audit-logs" -H "Authorization: Bearer $PAT" | jq '.data[0:3]'

# 4. Burst test: for i in $(seq 1 210); do curl -o /dev/null -w "%{http_code}\n"
https://dp/; done
```

Lab Steps

1. Create global rate-limiting plugin via Konnect Admin API (see command on left)
2. Verify plugin is global: GET /core-entities/plugins — confirm service and route are null
3. Burst test: send 210 requests — confirm 429 on request 201+
4. Configure Audit Log SIEM export via Admin API: PUT /v2/audit-log-webhook
5. Verify: make a config change, query GET /v2/audit-logs — confirm entry appears
6. Add global http-log or prometheus plugin via Admin API for external monitoring

DOMAIN 06 — RESILIENCE & INFRASTRUCTURE

Hardening Is Meaningless If the Platform Falls Over

#6 Secure the Konnect-Provisioned DP Certificate

When creating a DP node, download the pinned TLS cert and key from Konnect (Gateway Manager → Data Planes). Store them in a Kubernetes Secret or Vault — never as plaintext files on disk. Know the cert expiry and reissue via the Konnect UI before it lapses.

#21 Manage DP Config as Code with decK

Use decK (declarative Kong configuration) to manage services, routes, and plugins as version-controlled YAML. Apply changes via CI/CD pipeline. Prevents config drift across DP nodes and provides a full audit trail for every configuration change.

#22 Appropriate HA Configuration

Minimum 2 DP nodes behind a load balancer in each deployment region. Kong manages CP availability — your responsibility is data plane redundancy. Size for one DP node failing without service degradation.

Konnect Advantage: Kong manages CP HA, CP cert rotation, and platform upgrades. Your hardening scope is the Data Plane — which is exactly where all of today's controls live.

COMPLETE HARDENING REFERENCE

Your 24-Control Checklist

- [ ] 01 Disable HTTP everywhere
- [ ] 02 LDAP/IdP credentials via secrets manager
- [ ] 03 Consumer credential secrets via Vault
- [ ] 04 PKI mode for DP
- [ ] 05 DP health checks at LB
- [ ] 06 Konnect DP cert stored securely (K8s Secret/Vault)
- [ ] 07 RBAC — least privilege via Konnect Teams
- [ ] 08 Key-auth encrypted + Vault-backed
- [ ] 09 DP nodes isolated in private VPC/subnet
- [ ] 10 Disable anonymous reporting
- [ ] 11 Konnect org access IP-restricted
- [ ] 12 Server fingerprinting disabled
- [ ] 13 Debug header disabled
- [ ] 14 TLS for all integrations
- [ ] 15 Minimum TLS 1.2 enforced
- [ ] 16 Nginx access logs off (DP)
- [ ] 17 Real IP configured
- [ ] 18 Konnect Analytics exported to monitoring
- [ ] 19 Konnect Audit Logs exported to SIEM
- [ ] 20 Untrusted Lua execution disabled
- [ ] 21 MFA enabled
- [ ] 22 HA — 2+ DP nodes per region behind LB
- [ ] 23 Global observability plugins
- [ ] 24 Global rate limiting plugins

KEY TAKEAWAYS

What to Do on Monday Morning

01

HTTP is not a configuration option — it's a liability.

Enforce HTTPS everywhere. Proxy, admin, dev portal. No exceptions for internal-only services. Your network perimeter is not your security boundary anymore.

02

A secret in a config file is already leaked.

Every credential in a DP environment variable or Kubernetes Secret that isn't Vault-backed is one misconfigured RBAC away from exposure. Vault integration is a one-day task.

03

Rate limiting and audit logging are not optional extras.

Global rate limiting is your DDoS floor. Audit logging is your forensic record. Both need to be on before you go to production — not after your first incident.

04

Hardening is a posture, not a one-time task.

This checklist is a starting point. Run it quarterly. Every Kong upgrade, every new workspace, every new team member is a reason to re-validate your posture.

05

Konnect takes the CP burden off your team.

Konnect manages control plane HA, upgrades, and cert rotation for you. You own the data plane — which is where most of these controls live. Focus your effort there.

RESOURCES & NEXT STEPS

Questions?

- **Konnect Security & Network Resiliency**
  docs.konghq.com/konnect/network-resiliency/
- **Konnect Teams & Roles (RBAC)**
  docs.konghq.com/konnect/org-management/teams-and-roles/
- **Konnect Configuration Docs**
  docs.konghq.com/konnect/
- **Grafana Dashboard #7424**
  grafana.com/grafana/dashboards/7424
- **OWASP API Security Top 10**
  owasp.org/www-project-api-security/

Feedback

https://konghq.surveymonkey.com/r/7XLD89X

Thank you !!!

Make sure to provide feedback and let us know what you want to hear next.

https://konghq.surveymonkey.com/r/7XLD89X